

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

VI. GESTION DES EXCEPTIONS

VI. GESTION DES EXCEPTIONS

Exemple

```
package exceptions;
public class Test1 {
    public static void main(String[] args) {
        int a=4;
        int b=2;
        System.out.println(a+" "+b);
        System.out.println(diviser(a,b));
        System.out.println("Fin du programme.");
    }
    public static int diviser(int a, int b) {
        return a/b;
    }
}
```

```
4 2
2
Fin du programme.
```

VI. GESTION DES EXCEPTIONS

Exemple

```
package exceptions;
public class Test1 {
    public static void main(String[] args) {
        int a=4;
        int b=0;
        System.out.println(a+" "+b);
        System.out.println(diviser(a,b));
        System.out.println("Fin du programme.");
    }
    public static int diviser(int a, int b) {
        return a/b;
    }
}
```

L'exécution est interrompue
L'exception levée est
java.lang.ArithmeticException :
Le message est: / ***by zero***
Cette exception est **sans**
paramètres.
La trace des appels des methods
amenant à l'erreur est affichée.

4 0

Exception in thread "main" java.lang.ArithmeticException: / by zero
at exceptions.Test1.diviser(Test1.java:11)
at exceptions.Test1.main(Test1.java:7)

VI. GESTION DES EXCEPTIONS

```
public static void main(String[] args) {  
    double S = 0;  
    Scanner sc = new Scanner(System.in);  
    System.out.println("Entrez la taille du tableau:");  
    int N = sc.nextInt();  
    double[] T = new double[N];  
    System.out.println("Entrez les éléments du tableau:");  
    for (int i = 0; i < N; i++) {  
        T[i+1] = sc.nextDouble();  
        S = S + T[i];  
    }  
    double M = S / N;  
    System.out.println("La somme des "+  
        +"éléments du tableau est:" +  
        +S + " et la moyenne est:" + M);  
}
```

L'exception levée est
java.lang.ArrayIndexOutOfBoundsException :
Le message est:
***Index 2 out of bounds for
length 2***
Cette exception est avec
paramètres (index =2).

Entrez la taille du tableau:

2

Entrez les éléments du tableau:

1

3

Exception in thread "main"

java.lang.ArrayIndexOutOfBoundsException: Index 2 out
of bounds for length 2 at
exceptions.SommeMoyenne.main(SommeMoyenne.java:14)

VI. GESTION DES EXCEPTIONS

Présentation

- Une exception est un événement indiquant que **quelque chose d'exceptionnel** (par exemple une erreur) est produite.
- Elle **interrompt le flot d'exécution** normal du programme.
- Exemples:
 - Division par zéro,
 - Un débordement de tableau ou de pile,
 - La lecture d'une donnée erronée,...
- Problème en C :
 - Prévoir une plage de valeurs dans la valeur de retour pour signaler les erreurs.
 - Propager les erreurs “manuellement”.
- En Java comme en C++, le mécanisme de remonté d'erreur est gérée par le langage.

VI. GESTION DES EXCEPTIONS

Gestion des exceptions avec try-catch

- Pour gérer les exceptions, il faut les rattraper (catch), cela va empêcher l'arrêt anormal du programme.
- Ca consiste à insérer un bloc try-catch:
 - Bloc **try**: contient le code qui provoque l'exception.
 - Bloc **catch**: contient le code à exécuter en cas d'exception.
 - Le **code qui suit le bloc catch sera exécuté dans tous les cas**, qu'il y ait ou pas d'exception
- Donc il n'y aura pas d'arrêt anormal.

VI. GESTION DES EXCEPTIONS

Structure de gestion d'exceptions

- Pour la gestion des exceptions, on utilise le bloc try... catch... finally. Sa syntaxe est:

```
try {  
    // Code pouvant entraîner une exception  
}  
catch(Exception e) {  
    // Code prenant en compte une exception i.e. un objet de  
    // la classe Exception ou d'une classe dérivée.  
}  
finally {  
    // Le code d'un bloc try qui sera exécuter en dernier lieu.  
}
```



```

public static void main(String[] args) {
    try{
        int a=4; int b=0;
        System.out.println(a+" "+b);
        System.out.println(diviser(a,b));
        System.out.println(a+"/"+b+"="+diviser(a,b));
    }
    catch( ArithmeticException e){
        System.err.println("Il y a eu une erreur de division."); // affiché en rouge
        System.out.println(e.getClass().getName()); // afficher le nom de l'exception
        System.out.println(e.getMessage()); //afficher le message de l'exception
        System.out.println(e); //afficher toString() de l'exception
        e.printStackTrace(); // afficher le stack qui a généré l'exception!
    }
    finally {
        System.out.println("Fin du programme.");
    }
}

public static int diviser(int a, int b) {
    return a/b;}

```

4 0

Il y a eu une erreur de division.

java.lang.ArithmeticException

/ by zero

java.lang.ArithmeticException: / by zero

java.lang.ArithmeticException: / by zero

at exceptions.Test1.diviser(Test1.java:27)

at exceptions.Test1.main(Test1.java:8)

Fin du programme.

VI. GESTION DES EXCEPTIONS

Traitement des erreurs

- En Java, on peut afficher des résultats en utilisant *System.out* et les erreurs en utilisant *System.err* .
- Pour les **méthodes** Java, il y a **deux possibilités** pour « **sortir** » de l'exécution:
 - La **voie normale**, avec un éventuel retour de résultat.
 - La **voie exceptionnelle** gérée par un mécanisme spécifique, qui est le mécanisme des exceptions.
 - Ce mécanisme permet de **définir par programme des conditions d'erreurs**.
 - Il est **pleinement intégré à JVM** pour récupérer des erreurs d'exécution à savoir des divisions par zéro, des erreurs d'index dans un tableau, ...

VI. GESTION DES EXCEPTIONS

Structure de gestion d'exceptions

- Lorsque le bloc **try** est exécuté:
 - 1) Si **aucune exception** est lancée dans ce bloc:
 - Le code du bloc catch n'est pas exécuté;
 - L'exécution continue avec le bloc suivant.
 - 2) Si une **exception est lancée**:
 - L'exécution du try est interrompue;
 - Le bloc du catch est exécuté;
 - L'exécution continue avec le bloc suivant de catch.

VI. GESTION DES EXCEPTIONS

Exercice

Quel est le résultat du programme suivant?

```
public class TestException4 {  
    public static void main(String[] args) {  
        int i = 3;  
        int j = 0;  
        try {  
            System.out.println("résultat = " + (i / j));  
        } catch (Exception e) {  
            System.out.println("Exception : " + e);  
        }  
        System.out.println("Fin du main");  
    }  
}
```

```
Exception : java.lang.ArithmeticException: / by zero  
Fin du main
```

Quel est le résultat du programme suivant?

```
public class TabException1 {  
    int[] tabEnt = new int[4];  
  
    void initTableau () {  
        try {  
            tabEnt[0] = 1; tabEnt[1] = 2;  
            tabEnt[2] = 3; tabEnt[3] = 4;  
  
            tabEnt[4] = 5;  
        } catch (Exception e) {  
            System.out.println ("Exception : " + e);  
        }  
        System.out.println("Fin de initTableau");  
    }  
}
```

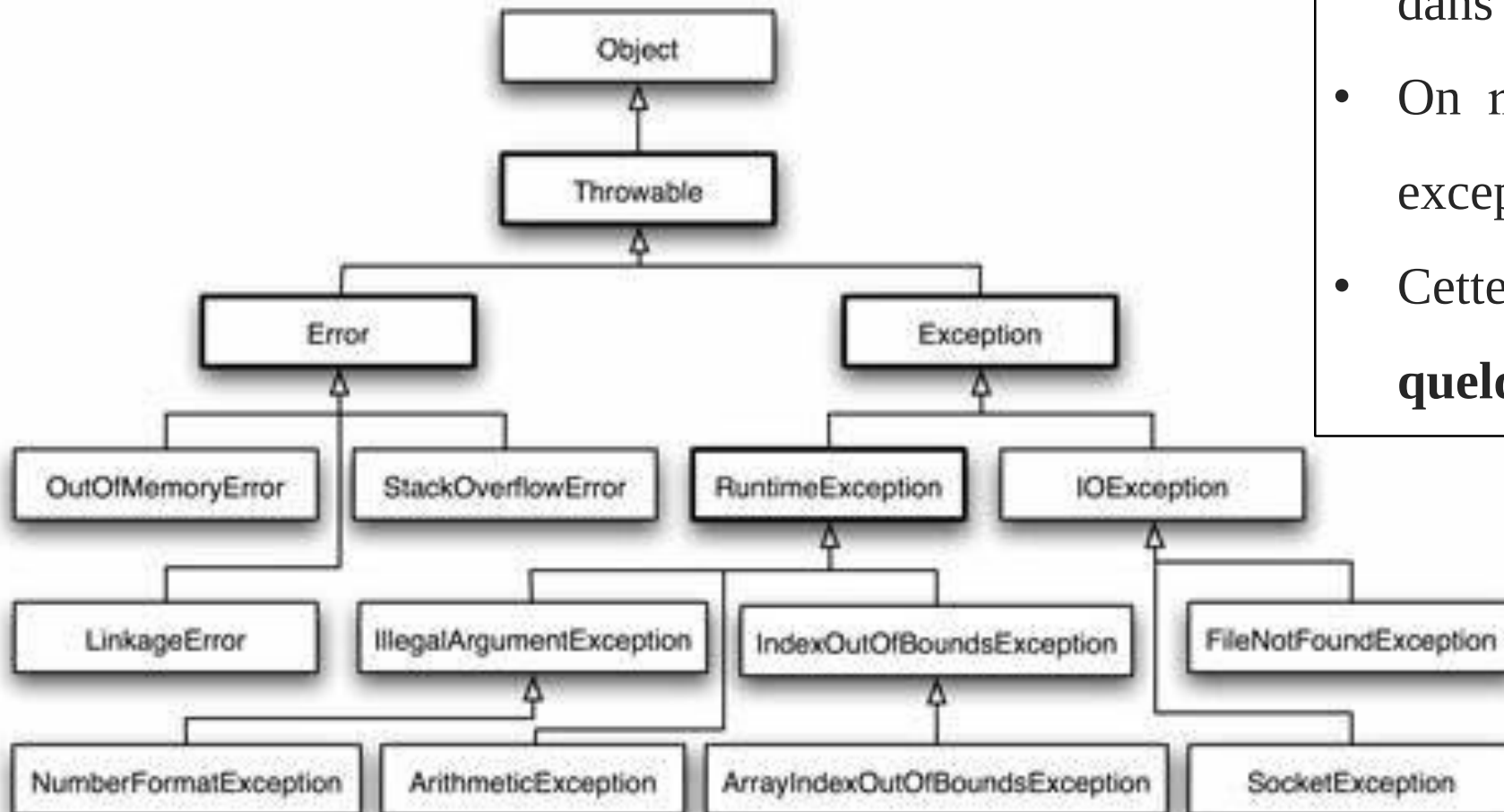
```
public class TestTabException {  
    public static void main(String[] args) {  
        TabException1 A = new TabException1();  
        A.initTableau();  
        A.afficheTableau();  
        System.out.println("Fin de main");  
    }  
}
```

```
Exception : java.lang.ArrayIndexOutOfBoundsException:  
Index 4 out of bounds for length 4  
Fin de initTableau  
  1 2 3 4  
Fin de main
```

```
void afficheTableau() {  
    for (int i = 0; i < tabEnt.length; i++) {  
        System.out.print(" " + tabEnt[i]);  
    }  
    System.out.println();}}
```

VI. GESTION DES EXCEPTIONS

Hiérarchie des exceptions



- Les exceptions Java sont définies dans **java.lang**.
- On n'a pas besoin d'importer ces exceptions.
- Cette illustration montre seulement **quelques exceptions** Java.

VI. GESTION DES EXCEPTIONS

Throwable: Quelques méthodes

public String getMessage()

Returns the detail message string of this throwable.

public void printStackTrace()

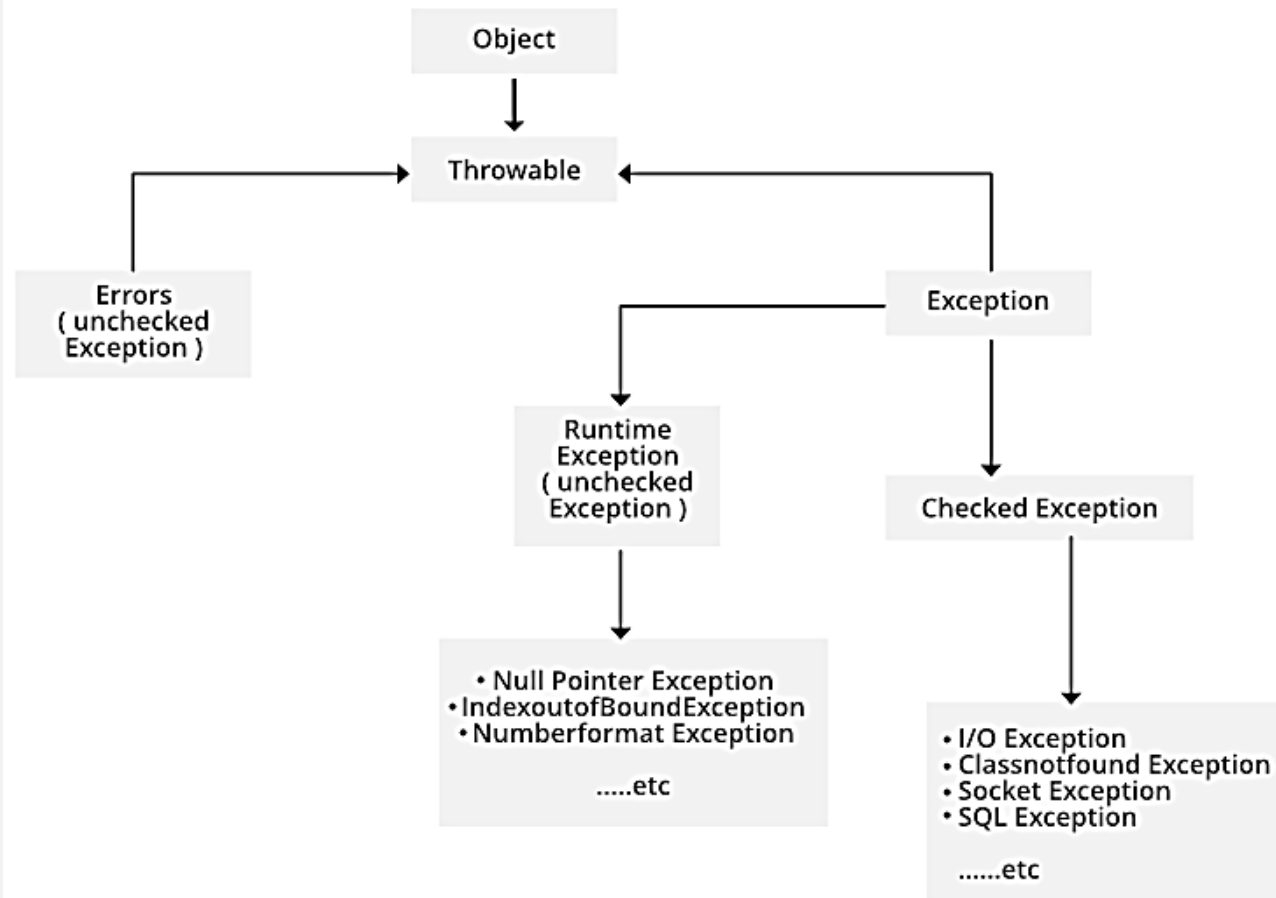
This method prints a stack trace for this Throwable object on the error output stream that is the value of the field `System.err`.

public String toString()

Returns a short description of this throwable.

VI. GESTION DES EXCEPTIONS

Hiérarchie des exceptions



VI. GESTION DES EXCEPTIONS

Hiérarchie des exceptions

- **Error** : décrit les erreurs internes ou le manque des ressources dans le système d'exécution de Java (**non récupérable=Unhandled**).
- **Exception (récupérable=Handled)**: est séparée également en deux branches, les exceptions dérivant de **RuntimeException(Unchecked)** et les autres (**Exceptions contrôlées=Checked**).
- **RuntimeException** se produit toujours en raison d'une erreur de programmation. Elle indique un problème d'exécution souvent imprévisible, comme : un mauvais transtypage., un accès à un tableau en dehors des limites.
- **Exemples:**
 - **NullPointerException** : tenter d'accéder à un objet ou un tableau de valeur nulle.
 - **ArithmeticException** : division entière par zéro.
 - **ArrayIndexOutOfBoundsException** : essai d'accès à un indice n'existant pas dans le tableau.
 - **ClassCastException** : transtypage impossible à l'exécution.

VI. GESTION DES EXCEPTIONS

Hiérarchie des exceptions

- **Exceptions contrôlées** : ce type d'exceptions a une cause externe à votre programme, comme :
 - **ClassNotFoundException**: la demande de chargement dynamique d'une classe est échouée à l'exécution.
 - **InterruptedException**: interruption d'un Thread.
 - **java.io.FileNotFoundException**: fichier non trouvé.
 - **java.net.MalformedURLException**: erreur syntaxique de description d'une URL.
 - **java.beans.PropertyVetoException**: le refus de la modification d'une propriété par une instance chargée de donner son avis.

VI. GESTION DES EXCEPTIONS

Exemple: Unchecked exception

Débordement de tableaux (*dans RuntimeException*)

```
public class TestException1 {  
    public static void main(String[] args) {  
        int tabEnt[] = { 1, 2, 3, 4 };  
        tabEnt[4] = 0; // erreur et arrêt  
        // Exception de type java.lang.ArrayIndexOutOfBoundsException  
        // non contrôlée (unhandled) par le programme; erreur d'exécution  
        System.out.println("Fin du main"); // ce message n'apparaît pas  
    }  
}
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index  
4 out of bounds for length 4  
    at exceptions.TestException1.main(TestException1.java:6)
```

VI. GESTION DES EXCEPTIONS

Exemple: Unchecked exception

Division par zéro (*dans RuntimeException*)

```
public class TestException2 {  
    public static void main(String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
        System.out.println("Fin du main"); // le message n'apparaît pas  
    }  
}
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at exceptions.TestException1.main(TestException1.java:7)
```

VI. GESTION DES EXCEPTIONS

Exemple: Unchecked exception

Débordement de tableaux (*dans RuntimeException*)

```
public class TestException3 {  
    public static void main(String[] args) {  
        try {  
            int tabEnt[] = { 1, 2, 3, 4 };  
            //Exception de type ArrayIndexOutOfBoundsException contrôlée par le programme  
            tabEnt [4] = 0; // l'exception se produit ici, et passe au catch  
            System.out.println ("Fin du try");//non exécutée  
        }  
        catch (Exception e) {  
            System.out.println ("Exception : " +e);//Affichage du message précédent  
        }  
    }  
}
```

Exception : `java.lang.ArrayIndexOutOfBoundsException`: Index 4 out of bounds for length 4

VI. GESTION DES EXCEPTIONS

Exemple: Checked exception

Fichier non trouvable (*dans IOException*)

```
public static void main(String[] args) {  
    FileReader file = new FileReader("somefile.txt"); // Erreur de compilation  
}
```

```
public static void main(String[] args) {  
    try {  
        FileReader file = new FileReader("somefile.txt");  
    }  
    catch (FileNotFoundException e1) {  
        e1.printStackTrace();  
    }  
}
```

VI. GESTION DES EXCEPTIONS

Gestion des exceptions

- Java est capable de générer tout type d'exceptions pour rendre compte des incidents spécifiques à chaque situation.
- Le programmeur pouvant ainsi **définir** ses propres **exceptions**, les **lancer** (**throws**) et les **récupérer** (**try..catch**).
- En général, quand un programmeur écrit un code lançant une exception, le compilateur s'assure qu'il y a vraiment un code permettant de récupérer l'incident.
- Sinon, on remonte jusqu'au sommet de la pile et l'exception se traite par un mécanisme par défaut.
- Ceci permet d'écrire un diagnostic dans la sortie d'erreur standard.

VI. GESTION DES EXCEPTIONS

Les exceptions d'utilisateurs

1. **Définition d'une exception:** Une classe d'exception est définie en **héritant** de la classe **Exception** se trouvant déjà dans l'API java.
 - **Les conventions de nom:** Les noms des exceptions doivent se terminer avec **Exception**.
 - **Bonne pratique:** Dans une exception, créer un constructeur avec un paramètre String contenant le **message détaillé** de l'erreur produite. Comme ça, on pourrait utiliser la méthode `getMessage()`;

```
public class ErreurCoordException extends Exception{  
    public ErreurCoordException(String message) {  
        super(message);  
    }  
    public ErreurCoordException(int x, int y) {  
        super("Erreur de coordonnées "+x+" "+y);  
    }  
}
```


VI. GESTION DES EXCEPTIONS

Les exceptions d'utilisateurs

2. On indique le **type d'exceptions** pouvant être rencontrées dans une **méthode** de la classe Point par le mot clé **throws**.
3. On **lance** une exception par le mot clé **throw new TypeException (ErreurCoordException())**.
 - Le mot-clé “**throw**” permet de **lancer** une exception.
 - Le mot-clé “**Throws**” s'utilise au niveau de la **signature d'une méthode** pour préciser que celle-ci est susceptible de lancer une exception.

```
public class Point {  
    private int x, y;  
  
    public Point(int x, int y) throws  
        ErreurCoordException {  
        this.x = x;  
        this.y = y;  
        this.affiche();  
    }  
    public void affiche() throws  
        ErreurCoordException {  
        if (x < 0 || y < 0)  
            throw new ErreurCoordException("Erreur dans  
le point. Coordonnées inférieures à 0. ");  
        System.out.println("Je suis un point de  
coordonnées " + x + " " + y);  
    }  
}
```

VI. GESTION DES EXCEPTIONS

Les exceptions d'utilisateurs

4. Ensuite pour **récupérer une exception** lancée par une méthode, on utilise les instructions **try** et **catch** :

```
public class Test {  
    public static void main(String[] args) {  
        try { // Instructions qui peuvent avoir une exception déclenchée  
            Point a = new Point(1, 4);  
            a = new Point(-3, 0);  
        }  
        // gestion de l'exception  
        catch (ErreurCoordException e) {  
            System.out.println(e);  
            System.out.println(e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}
```

```
Je suis un point de coordonnées 1 4  
Erreur de construction!  
Erreur dans le point. Coordonnées  
inférieures à 0.  
Erreur de construction!  
at  
exceptions.Point.affiche(Point.java:15)  
at exceptions.Point.<init>(Point.java:10)  
at exceptions.Test.main(Test.java:8)
```

VI. GESTION DES EXCEPTIONS

Les exceptions d'utilisateurs

- **Si une exception n'est pas attrapée dans le bloc où elle a été lancée**, elle est transmise au bloc de niveau supérieur (récursivement).
- **Si une exception n'est pas attrapée dans la méthode qui la lance**, elle est propagée dans la méthode qui invoque cette dernière (récursivement).
- **Si une exception n'est jamais attrapée:**
 - Propagation jusqu'à la méthode `main()`;
 - Affichage des messages d'erreurs et la pile d'appels;
 - Arrêt de l'exécution du programme;

Quel est le résultat du programme suivant?

```
class ErrConstException extends Exception {  
    private int errval;  
  
    public ErrConstException(int valeur) {  
        errval = valeur;  
    }  
    public int getValeur() {  
        return errval;  
    }  
}
```

```
public class EntierNaturel {  
    private int n;
```

```
    public EntierNaturel(int m) throws ErrConstException{  
        if (m<0)  
            throw new ErrConstException(m);  
        n=m;  
    }
```

```
    public int getn() {return n;}  
}
```

```
public class TestExcept2 {  
    public static void main(String[] args) {  
        try {  
            EntierNaturel n1 = new EntierNaturel(20);  
            System.out.println("n1= " + n1.getn());  
            EntierNaturel n2 = new EntierNaturel(-12);  
            System.out.println("n2 = " + n2.getn());  
        } catch (ErrConstException e) {  
            System.out.println("Erreur de construction  
avec valeur " + e.getValeur());  
        }  
    }  
}
```

n1= 20

Erreur de construction avec valeur -12

VI. GESTION DES EXCEPTIONS

Gestion de plusieurs exceptions

- On peut attraper différentes exceptions qui sont **indépendantes** avec `catch(Exception1 | Exception2 e)`

```
catch (ArithmeticException | ArrayIndexOutOfBoundsException e) // Pas d'erreur
```

```
catch (IndexOutOfBoundsException | ArrayIndexOutOfBoundsException e) // Erreur  
ArrayIndexOutOfBoundsException est incluse dans IndexOutOfBoundsException
```

- On peut attraper une exception plus générale en l'ajoutant à la fin de ces catch comme suit:

```
catch (ArithmeticException | IndexOutOfBoundsException e) {  
}  
catch (Exception e) { // Pour attrapper une exception autre que  
    // ArithmeticException et ArrayIndexOutOfBoundsException  
}
```

VI. GESTION DES EXCEPTIONS

Quelque propriétés des exceptions

- Les exceptions sont des objets, donc elles contiennent:
- Attributs particuliers, Méthodes particulières, Attributs et méthodes standards: *String getMessage()*, *void printStackTrace()*
- Chaque bloc « **try** » est suivi d'un ou de plusieurs blocs de « **catch** » permettant d'attraper les exceptions, dont le type est spécifié, et d'exécuter le code spécifique.
- Un **seul bloc de catch peut être exécuté**, c'est le premier susceptible d'attraper l'exception.
- Chaque clause catch se déclare avec un argument de type Throwable ou classe dérivée de Throwable.
- Lorsqu'une exception est lancée dans le bloc try, **la première clause dont le type d'argument correspond à celui de l'exception lancée est appelée.**
- **Remarque:** l'ordre des blocs de « catch » est important

VI. GESTION DES EXCEPTIONS

Exercice

- On a une classe Article, caractérisée par le stock en cours et le stock maximal.
- Le stock est initialisé à 0 et le stock maximal doit avoir une valeur positive.
- Si je veux remplir le stock je dois m'assurer que ça ne va pas dépasser le stock maximal.
- Dans le test, ajouter à la fin une opération de type `int a=1/0`.
- Ecrire le programme pour gérer les articles avec toutes les exceptions possibles.

```
public class StockNegatifException extends Exception{  
  
    public StockNegatifException(int stock) {  
        super("Vous avez saisi un stock négatif "+stock);  
    }  
}
```

```
public class StockException extends Exception{  
  
    public StockException(int stock) {  
        super("Vous avez dépassé le stock maximal:"+stock);  
    }  
}
```



```
public class Article {  
    private int stockMax;  
    private int stock;  
  
    public Article(int stockMax) throws StockNegatifException{  
        if(stockMax<0) throw new StockNegatifException(stockMax);  
        stock=0;  
        this.stockMax = stockMax;  
    }  
  
    public void remplirStock(int stock) throws StockException{  
        int stockR=this.stock+stock;  
        if(stockR>stockMax) throw new StockException(stock);  
        this.stock=stockR;  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Article a1;  
        try {  
            a1 = new Article(-1);  
            a1.remplirStock(12);  
            int k=1/0;  
        }  
        catch (StockException | StockNegatifException e) {  
            System.out.println(e.getMessage());  
            System.out.println(e);  
            e.printStackTrace();  
        }  
        catch(Exception e) {  
            System.out.println(e.getMessage());  
            System.out.println(e);  
            e.printStackTrace();  
        }  
    }  
}
```

Vous avez saisi un stock négatif -1
Exception dans le stock maximal.
Exception dans le stock maximal.
at
exceptions.myExample.Article.<init>([Article.java:8](#))
at exceptions.myExample.Test.main([Test.java:7](#))

```

public class Test {
    public static void main(String[] args) {
        Article a1;
        try {
            a1 = new Article(10);
            a1.remplirStock(12);
            int k=1/0;
        }
        catch (StockException | StockNegatifException e) {
            System.out.println(e.getMessage());
            System.out.println(e);
            e.printStackTrace();
        }
        catch(Exception e) {
            System.out.println(e.getMessage());
            System.out.println(e);
            e.printStackTrace();
        }
    }
}

```

Vous avez dépassé le stock maximal:12
 Exception dans le remplissage du stock.
 Exception dans le remplissage du stock.
 at exceptions.myExample.Article.remplirStock([Article.java:18](#))
 at exceptions.myExample.Test.main([Test.java:8](#))

```

public class Test {
    public static void main(String[] args) {
        Article a1;
        try {
            a1 = new Article(10);
            a1.remplirStock(9);
            int k=1/0;
        }
        catch (StockException | StockNegatifException e) {
            System.out.println(e.getMessage());
            System.out.println(e);
            e.printStackTrace();
        }
        catch(Exception e) {
            System.out.println(e.getMessage());
            System.out.println(e);
            e.printStackTrace();
        }
    }
}

```

/ by zero

java.lang.ArithmeticException: / by zero

java.lang.ArithmeticException: / by zero

at exceptions.myExample.Test.main(Test.java:9)